

Day 41\* Local Variables and Global Variables :-• Local Variables :-

- The variable used inside of function body are called "Local variables".
- The purpose of local variables is that "to store the temporary results".

Syntax :    def functionname (list of formal params if any):

var1 = val1

var2 = val2

var-n = val-n

⇒ Here, var1, var2, ... var-n are called local variables.

• Global Variables :-

- Global variables are those which are common values for different function calls.
- In other words, if the value is common for all different functions calls then such type of values must be taken as Global variables.
- To access the values of Global variables then they must be defined before function calls only otherwise we get name error.

Syntax :

var1 = val1

var2 = val2

var-n = val-n

def fun1():

def fun2():

def fun3():

⇒ Here var1, var2, ... var-n are called Global variables and we can access those values inside of fun1(), fun2(), ... fun-n().

# Program for demonstrating local and global variable.

Ex 1. lang = "PYTHON" # Global variable.

```
def learnML():
    sub1 = "ML"
    print("to develop 'ML' programs, we use 'ML' lang.")
    format(sub1, lang)
    # print(sub2) can't access bcoz sub2 is local
    in learnML()

def learnDL():
    sub2 = "DL"
    print("to develop 'DL' programs, we use 'DL' lang.")
    format(sub2, lang)
    # print(sub1) can't access bcoz sub1 is local in learnML()

# main program.
learnML()
learnDL()
=.
```

Ex 2. Bas wahan pe lang = "PYTHON" # Global variable

Ex 3. Is example Binga lang define krke.

```
# main program
lang = "python" # Global variable
```

Ex 4. # main program

```
# learnML() --- Here global variable lang can't be
accessed in its body bcoz it was defined
after its function call.

lang = "python" # Global variable
```

### \* global keyword :-

→ when we want to modify the GLOBAL VARIABLE values inside of function definition then global variable names must be preceded with 'global' keyword otherwise we get "Unbound local error". local variable names referenced before assignment?

Syntax :

Var1 = Val1

Var2 = Val2

Var-n = Val-n      # Var1, Var2 ... Var-n are called global variable names

def fun1():

    global Var1, Var2 ... Var-n

    # modify Var1, Var2 ... Var-n

def fun2():

    global Var1, Var2 ... Var-n

    # modify Var1, Var2 ... Var-n

# program for demonstrating global keyword.

def incr1():

    global a

    a = a + 1

def incr2():

    global a

    a = a \* 2

# main program

a = 10    # global variable

print("Val of a in main program before incr1() =", a)    # 10

incr1()

print("Val of a in main program after incr1() =", a)    # 11

incr2()

print("Val of a in main program after incr2() =", a)    # 22

Ex 10.

```
def update():
    global a, b;
    a = a + 1
    b = b + 1

# main program
a = 10
b = 20 # Here a & b are called global variables.
print("In main program before update() --> a = {} and b = {}".format(a, b))
update()
print("In main program before update() --> a = {} and b = {}".format(a, b))
```

Ex 11.

```
def update():
    # no need to write global keyword bcoz we are updating global variables a & b values and just we are accessing
    c = a + 1
    d = b + 1
    print("In update() c = {} and d = {}".format(c, d))

# main program
baaki sab same.
```

### \* Global and Local Variables and globals()

→ When we come across same global variable names and local variable names in same function definition then python gives preference to local variables but not for global variables.

→ In this context, to extract/retrieve the values of global variables names along with local variables, we must use `globals()` and it returns an object of `<class, 'dict'>`

↓  
and this dict object stores all global variables names as keys and global variable values as values of value.



Syntax :

```

Var1 = val1
Var2 = val2
...
Var-n = val-n # Here Var1, Var2 ... Var-n are called
                global variables.

def functionname():
    var1 = val1
    var2 = val2
    ...
    Var-n = val-n # Var1, Var2 ... Var-n are called
                    local variables.

# Extract the global variables values.
dictobj = globals()
global val1 = dictobj['var1'] # or
# dictobj.get('var1') or globals()['var1']
# or global().get('var1')
global val2 = dictobj['var2']
=

```

# program for demonstrating globals()

Ex 10

```

a = 10
b = 20
c = 30
d = 40 # Here a, b, c and d are called global variable

```

def operations():

x = 100

y = 200

z = 300

k = 400

# Here x, y, z, k are called local variables

res = x + y + z + k + a + b + c + d.

print("Result =", res)

# main program

operations()Ex 10

x, y, z, k, hasil a, b, c, d.

```

res = a + b + c + d + globals()['a'] + globals()['b'] + globals()['c']
      + globals()['d']

```

Ex ① :-a=10  
b=20

# Here a, b are called global variables.

def operations():

dictobj = globals()

print("Both grammar and Implicit Global variables:")

for gvn, gvv in dictobj.items():

print("{} {} --&gt; {}".format(gvn, gvv))

print(" - - - - -")

print("programmer-defined global variable-way")

print("val of a:", dictobj['a'])

print("val of b:", dictobj['b'])

print(" - - - - -")

By change Hope dictobj.get('a')

globals().get('a')

globals()['a']

# main program

operations()# Read valuesex ①. lst = [] # empty list

n = int(input("Enter How many value you want in list:"))

if (n &lt;= 0):

print("{} invalid input".format(n))

else:

for i in range(1, n+1):

val = float(input("Enter {} value:".format(i)))

lst.append(val)

else:

print("List of values: {}".format(lst))

ex ②. print("Enter list of values separated by space:")

lst = [float(val) for val in input().split()]

print("Content of list =", lst)

## local variables and global variables

```
In [5]: #program for demonstrating local and global variables
#ex1:
lang="python"           #global variables
def learnml():
    sub1="ml"
    print("to devleop '{}' programs,we use '{}' lang".format(sub1,lang))
    #print(sub2) can't access bcoz sub 2 is local variables
def learndl():
```

```

    sub2="dl"
    print("to develop '{}' programs,we use '{}' lang".format(sub2,lang))
    #print(sub1) can't access bcoz sub 2 is local variables
#main program
learnml()
learndl()

```

to devleop 'ml' programs,we use 'python' lang  
to develop 'dl' programs,we use 'python' lang

```

In [12]: #program for demonstrating local and global variables
#ex2:
lang="python"           #global variables
def learnml():
    sub1="ml"
    lang="python"
    print("to devleop '{}' programs,we use '{}' lang".format(sub1,lang))
    #print(sub2) can't access bcoz sub 2 is local variables
    lang="python"
def learndl():
    sub2="dl"
    print("to develop '{}' programs,we use '{}' lang".format(sub2,lang))
    #print(sub1) can't access bcoz sub 1 is local variables
#main program
learnml()
learndl()

```

to devleop 'ml' programs,we use 'python' lang  
to develop 'dl' programs,we use 'python' lang

```

In [13]: #program for demonstrating local and global variables
#ex3:
def learnml():
    sub1="ml"
    print("to devleop '{}' programs,we use '{}' lang".format(sub1,lang))
    #print(sub2) can't access bcoz sub 2 is local variables
def learndl():
    sub2="dl"
    print("to develop '{}' programs,we use '{}' lang".format(sub2,lang))
    #print(sub1) can't access bcoz sub 2 is local variables
#main program
lang="python"
learnml()
learndl()

```

to devleop 'ml' programs,we use 'python' lang  
to develop 'dl' programs,we use 'python' lang

```

In [16]: #program for demonstrating local and global variables
#ex4:
def learnml():
    sub1="ml"
    print("to devleop '{}' programs,we use '{}' lang".format(sub1,lang))
    #print(sub2) can't access bcoz sub 2 is local variables
def learndl():
    sub2="dl"
    print("to develop '{}' programs,we use '{}' lang".format(sub2,lang))
    #print(sub1) can't access bcoz sub 2 is local variables
#main program
#Learnml()    #-----here global variables lang can't be accessed in its body bco

```

```
learndl()
lang="python"
```

to develop 'dl' programs, we use 'python' lang

## global keyword

```
In [17]: #program for demonstrating global keyword
#ex-1
def incr1():
    global a
    a=a+1
def incr2():
    global a
    a=a*2
#main program
a=10 #global variables
print("val of a in main program before incr1()=",a)
incr1()
print("val of a in main program after incr1()=",a)
incr2()
print("val of a in main program after incr2()=",a)
```

val of a in main program before incr1()= 10

val of a in main program after incr1()= 11

val of a in main program after incr2()= 22

```
In [18]: #program for demonstrating global keyword
#ex-2
def update():
    global a,b
    a=a+1
    b=b+1
#main program
a=10
b=20
print("in main program before update1()----->a={} and b={}".format(a,b))
update()
print("in main program before update1()----->a={} and b={}".format(a,b))
```

in main program before update1()----->a=10 and b=20

in main program before update1()----->a=11 and b=21

```
In [25]: #program for demonstrating global keyword
#ex-3
def update1():
    c=a+1
    d=b+1
    print("in update1() c={} and d={}".format(c,d))
#main program
a=10
b=20
print("in main program before update1()----->a={} and b={}".format(a,b))
update1()
print("in main program before update1()----->a={} and b={}".format(a,b))
```

in main program before update1()----->a=10 and b=20

in update1() c=11 and d=21

in main program before update1()----->a=10 and b=20



## global and local variables and globals()

```
In [32]: #program for demonstrating globals()
#ex1:
a=10
b=20
c=30
d=40
def operations():
    x=100
    y=200
    z=300
    k=400
    res=x+y+z+k+a+b+c+d
    print("result ",res)
#main program
operations()
```

result 1100

```
In [41]: #program for demonstrating globals()
#ex2:
a=10
b=20
c=30
d=40
def operations():
    a=100
    b=200
    c=300
    d=400
    res=a+b+c+d+ globals()['a']+globals()['b']+globals()['c']+globals()['d']
    print("result",res)
#main program
operations()
```

result 1100

```
In [43]: # read values
#ex1:
lst=[]
n=int(input("enter how many value you want to insert:"))
if(n<=0):
    print("{} invalid input".format(n))
else:
    for i in range(1,n+1):
        val=float(input("enter{} value:".format(i)))
        lst.append(val)
    else:
        print("list of values :{}".format(lst))
```

list of values :[4.0, 5.0, 8.0, 9.0]

```
In [44]: #read values
#ex2:
print("enter list of values separated by space:")
lst=[float(val) for val in input().split()]
print("content of list=",lst)
```

enter list of values separated by space:

content of list= [4.0]

```
In [45]: #program for demonstrating globals()
#ex3:
a=10
b=20
# here a,b are called global variables
def operations():
    dictobj=globals()
    print("Both Programmer and Implicit Global variables:")
    for gvn,gvv in dictobj.items():
        print("\t{ }-->{ }".format(gvn,gvv))
    print("-----")
    print("Programmer-Defined Global variables-way1")
    print("Val of a:", dictobj['a'])
    print("Val of b:", dictobj['b'])
    print("-----")
    print("Programmer-Defined Global variables-way2")
    print("Val of a:", dictobj.get('a'))
    print("Val of b:", dictobj.get('b'))
    print("-----")
    print("Programmer-Defined Global variables-way3")
    print("Val of a:", globals().get('a'))
    print("Val of b:", globals().get('b'))
    print("-----")
    print("Programmer-Defined Global variables-way4")
    print("Val of a:", globals()['a'])
    print("Val of b:", globals()['b'])
    print("-----")
#main program
operations()
```